Self-Supervised Spectral–Spatial Graph Prototypical Network for Few-Shot Hyperspectral Image Classification

Shan Ma, Lei Tong, *Member, IEEE*, Jun Zhou, *Senior Member, IEEE*, Jing Yu, and Chuangbai Xiao

***Abstract*— In recent years, deep learning has been widely applied to hyperspectral image (HSI) classification (HIC) with great success. However, since labeling hyperspectral images (HSIs) is time-consuming and labor-intensive, a limited number of labeled HSIs are available, making it difficult to train feature extractors and classifiers. To address this challenge, this article introduces a self-supervised spectral–spatial graph prototypical network (S^{4}GPN) for few-shot HIC. In addition, we combine self-supervised learning (SSL) with few-shot learning (FSL) to provide additional semantic information to improve classification accuracy. Our method consists of three stages, including the prototypical network (PN) stage, the SSL stage, and the fusion stage, with each stage progressively improving the classification performance. In the PN stage, we perform supervised learning using a PN structure and leverage the supervised information to guide SSL. In the SSL stage, we use the SimSiam structure in a novel way for model training after the augmentation of spectral and spatial data separately. Finally, the features learned in the first two stages are fused to improve the quality of the feature representation. These three stages use the same structured feature extractors. To extract more diverse and discriminative feature representations for the HIC task, our method uses the graph convolution network (GCN) and the dense network (Densenet) to extract spectral information and spatial information, respectively. Experiments on four datasets and comparisons with the state-of-the-art methods demonstrate that our proposed S^{4}GPN outperforms other methods for HIC.**

***Index Terms*— Few-shot learning (FSL), hyperspectral image (HSI) classification (HIC), prototypical network (PN), self-supervised learning (SSL).**

I. Introduction

HYPERSPECTRAL images (HSIs) usually contain information on objects in hundreds of continuous spectral bands. As remote sensing technology develops rapidly, HSIs have also been greatly improved in terms of spatial resolution, which enhances the ability of hyperspectral data to represent image features [1]. The unity of spatial data and the wide range of wavelengths can further improve the ability of HSIs to distinguish and identify features, making hyperspectral remote sensing technology widely used in agriculture, geological exploration, and other applications [2], [3], [4], [5], [6], [7], [8].

Among various remote sensing tasks, hyperspectral image (HSI) classification (HIC) is to determine the corresponding class of each pixel in the image [9], [10], [11]. Most traditional HIC methods, such as support vector machines (SVMs) [12], [13], Bayesian classifiers [14], random forests [15], extreme learning machines (ELMs) [16], sparse representation [17], and maximum likelihood classifiers [18], are based only on the 1-D spectral information while ignoring their 2-D spatial information. Markov random field (MRF) [19] and some spectral–spatial kernel methods [20], [21] introduce 2-D spatial information to improve the classification results; however, these traditional classification methods rely on handcraft feature extraction, rich feature design experience, and domain knowledge.

In recent years, deep learning algorithms have been intensively applied to HIC. Stacked autoencoder (SAE) [22] and deep belief network (DBN) [23] are among the early adopted deep learning networks with fully connected neural network structures. Convolutional neural network (CNN) methods have also become powerful tools for the representation of spectral information [24], joint spectral and spatial information [25], [26], [27], [28], [29], or spectral and spatial information separately [30], [31], [32]. Among them, the 3D-CNN proposed by Li et al. [25] extracts spectral–spatial information directly from HSI cube data without any preprocessing or postprocessing of the data. Gao et al. [28] proposed an end-to-end stochastic depth-based residual network (SDRN), which adopts a stochastic depth strategy to train spectral–spatial information separately through spectral residual blocks and spatial residual networks. The fast dense spectral–spatial convolution network framework (FDSSC) proposed by Wang et al. [31] uses different sizes of convolutional kernels to extract spectral and spatial features.

Although CNN-based methods have produced great performance, they still have a few drawbacks. These include, for example, sizes and shapes of the convolutional kernels of the CNN-based methods are fixed, the weights cannot be applied to all regions of the HSI, boundary features are lost during

Manuscript received 29 May 2023; revised 9 July 2023; accepted 13 July 2023. Date of publication 20 July 2023; date of current version 15 August 2023. This work was supported in part by the National Natural Science Foundation of China under Grant 61701009 and in part by the Beijing Municipal Education Commission Science and Technology Program under Grant KM202010005016. *(Corresponding author: Lei Tong.)*

Shan Ma, Lei Tong, Jing Yu, and Chuangbai Xiao are with the Faculty of Information Technology, Beijing University of Technology, Beijing 100124, China (e-mail: lei_tong@bjut.edu.cn).

Jun Zhou is with the School of Information and Communication Technology, Griffith University, Nathan, QLD 4111, Australia.

Digital Object Identifier 10.1109/TGRS.2023.3297216

1558-0644 © 2023 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 02:46:12 UTC from IEEE Xplore. Restrictions apply.

feature extraction, and due to the inflexible convolution kernel, potential misclassification is likely to occur. These problems can be partially alleviated by the graph convolutional network (GCN)-based HIC [33], [34], [35], [36], [37] which extracts features from the graph and then aggregates and transforms the feature information in each graph node to make full use of non-Euclidean irregular data based on the predefined graph structure. However, the supervised classifier still struggles to obtain satisfactory classification results due to the limited labeled samples and the possible overfitting problem of the deep network.

One solution to data and overfitting problems is few-shot learning (FSL) [38] which aims to develop a classifier with few labeled samples [39], [40], [41], [42]. Tang et al. [40] proposed a spatial–spectral prototypical network (SSPN) that measures the distances between the prototype and the query samples for classification. Liu et al. [39] proposed a deep FSL (DFSL), which learns a metric space from four source datasets via a deep residual network based on 3D-CNN, and then uses an SVM or the nearest neighbor (NN) methods to classify unlabeled data. The deep cross-domain FSL (DCFSL) proposed by Li et al. [42] unifies the FSL and cross-domain problems under the same framework. However, the information that can be obtained with limited samples is mainly focused on the differences in the base class samples, so it is impossible to obtain sufficient semantic information for all the classes.

In addition to FSL, self-supervised learning (SSL) [43], [44] can improve the representation ability of data for small sample problems. In the field of computer vision, among the various SSL algorithms, contrastive learning [45], [46], [47] has been studied intensively in recent years. Chen and He [48] explored simple Siamese representation learning (SimSiam), which is a typical method for contrast learning. It directly maximizes the similarity between two same images without the need for negative sample pairs, large batches, and momentum coding similarities of augmented views to learn representations. SSL is also widely applied in hyperspectral research. For example, the SSL by the adaptive distillation (SSAD) algorithm [49] implements SSL through the 3-D rotation transformation of the spectral–spatial domain of the original HSI and adaptive soft labeling.

Typically, SSL methods based on FSL use SSL as an auxiliary task to implement the main task of FSL. However, due to the scarcity of samples, the semantic information learned by traditional self-supervised methods may mainly focus on the differences between the base class samples and ignore the information valuable to the new class, which may lead to wrong guidance for classification. In this article, we propose a self-supervised spectral–spatial graph prototypical network (S^4GPN) to solve the above problem. The S^4GPN is a staged model that combines the advantages of supervised FSL and SSL. First, in the prototypical network (PN) stage, we perform supervised FSL using a PN structure. Then in the SSL stage, we use the SimSiam structure for SSL, in which different augmentations are applied to the spectral and spatial domains for the specificity of HSIs. We also use the supervised information obtained in the PN stage to guide SSL. Finally, in the fusion stage, we fuse the representations obtained from the first two training stages. In addition, GCN and dense network (Densenet) are used, respectively, to extract spectral and spatial features to fully exploit the overall HSI information.

The main contributions of our work are as follows: 1) to add additional semantic information on the basis of limited samples, we jointly use FSL and SSL to propose a three-stage framework; 2) we propose a novel self-supervised data augmentation method for spectral and spatial data; and 3) in all the three stages, we adopt the same feature extraction structure that uses GCN to extract spectral information and Densenet to extract spatial information.

The remainder of this article is organized as follows. In Section II, the related works are introduced briefly. In Section III, the proposed S^4GPN is described in detail. Section IV gives the experimental results and analysis. Finally, we conclude in Section V.

II. Related Work

In this section, we review PN, SSL, and GCN. They constitute the bases of our proposed S^4GPN.

A. Prototypical Network

PN belongs to FSL methods. In FSL, episodes T are used in the training process, and each episode T_i samples a task Γ_i. Each task contains a support set S_i and a query set Q_i. For train dataset $D_{tr} = \{\Gamma_i\}_{i=1}^{I} = \{S_i, Q_i\}_{i=1}^{I}$, the support set S_i is composed of $N \times K$ samples in which K samples are taken from each of the N classes. The query set Q_i is also composed of $N \times M$ samples, but the M samples for each class do not overlap with S_i. Similarly, the test dataset $D_{te} = \{\Gamma_i^*\}_{i=1}^{I} = \{S_i^*, Q_i^*\}_{i=1}^{I}$ consists of a support set S_i^* and a query set Q_i^* and does not overlap with the training set. FSL can be classified into three main types of methods: model-based fine-tuning, data-based augmentation, and migration-based learning. The migration-based methods can be further divided into three types: metric-based learning, graphical neural networks, and meta-learning. The PN used in this article is one of the metric-based methods [50].

The core idea of PN is to compute the embedding centers in the support set in an N-way, K-shot setting, and then measure the distance of the new samples (query set) from these centers to complete the classification. The support set contains a small number of annotated samples and the query set contains unannotated samples, which must be in the same sample space as the support set but not overlapped with the support set. For each classification, the embedding center (the prototype representation) $\mathbf{c}_k$ is calculated as follows:

$$\mathbf{c}_k = \frac{1}{|S_k|} \sum_i f(\mathbf{x}_i) \tag{1}$$

where S_k denotes the support set with class k, f denotes the embedding function, and $\mathbf{x}_i$ denotes the input. Then, the distance $\mathbf{d}_k$ from the new sample $\mathbf{x}^*$ to the embedding center of each class k is calculated as follows:

$$\mathbf{d}_k = d\big(f(\mathbf{x}^*), \mathbf{c}_k\big), \quad k = 1, \ldots, N \tag{2}$$

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 02:46:12 UTC from IEEE Xplore. Restrictions apply.

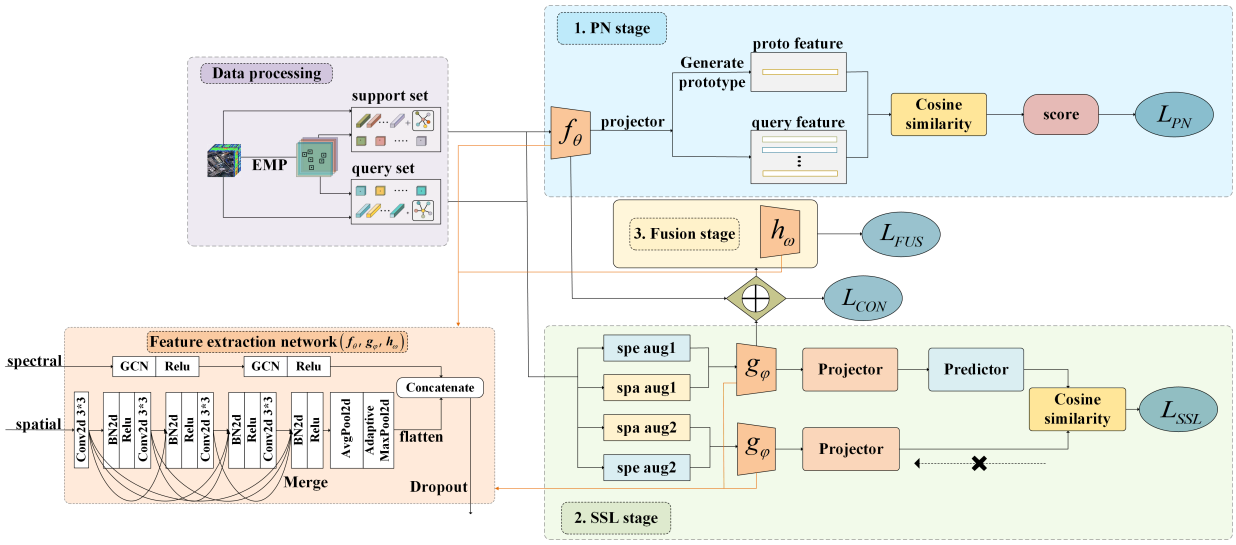


Fig. 1. Framework of S^{4}GPN. The method sends the processed data to the PN stage, SSL stage, and fusion stage in turn for training. The same feature extraction network is used in all three stages.

where $d(\cdot,\cdot)$ denotes the distance metric, usually using Euclidean distance, and $\mathbf{c}_k$ denotes the center of the embedding for class k. In the next step, the distance is mapped using softmax to obtain the probability $\hat{y}_k$ for each class

$$\hat{y}_k = \text{softmax}(\mathbf{d}_1, \ldots, \mathbf{d}_N). \tag{3}$$

Finally, the objectives are trained by a stochastic gradient descent (SGD) optimized cross-entropy loss function

$$L(\mathbf{y}, \hat{y}) = -\sum_{k=1}^{N'} \mathbf{y}_k \log \hat{y}_k. \tag{4}$$

B. Self-Supervised Learning

SSL [51] aims to use auxiliary tasks to mine large-scale unlabeled data for potential information and use the constructed supervised information to train the network to improve the feature extraction capability of the model. SSL methods can generally be classified into two types: generative method and discriminative method. The generative method takes the raw information of the input data, maps it into the hidden space, and then uses a generator to restore it to its original form. The discriminative method is more widely used since it differentiates the original data in a way that does not require data reconstruction at the pixel level. The discriminative method is achieved by constructing various different auxiliary tasks for SSL of unlabeled data, and a common discriminative method is contrastive learning. Contrastive learning uses the differences between data to learn, that is, by bringing positive examples closer to positive examples while further away from negative examples, thus providing a better representation of the data.

SimSiam [48], used in this article, is a common structure in contrastive learning. The main idea is to use a stop-gradient operation to truncate the gradient propagation on one side of the network to prevent model degradation. The SimSiam model takes two random augmentations $\mathbf{x}^1$ and $\mathbf{x}^2$ of image $\mathbf{x}$ as input. These two views are encoded by an encoder g to obtain z_1 and z_2, which consists of a backbone network and a multilayer perceptron. The encoder g shares the weights between the two views. The output z_1 is then passed through a predictive multilayer perceptron m as $\mathbf{p}_1$ (i.e., $\mathbf{p}_1 = m(g(\mathbf{x}^1))$ and $z_2 = g(\mathbf{x}^2)$). Their negative cosine similarity Ψ is then minimized, defining a symmetric loss function as follows:

$$L = \frac{1}{2}\Psi(\mathbf{p}_1, z_2) + \frac{1}{2}\Psi(\mathbf{p}_2, z_1) \tag{5}$$

where the negative cosine similarity Ψ is calculated as follows:

$$\Psi(\mathbf{p}_1, z_2) = -\frac{\mathbf{p}_1}{\|\mathbf{p}_1\|_2} \cdot \frac{z_2}{\|z_2\|_2}. \tag{6}$$

An important component of the backpropagation process of the backbone network training is the gradient truncation operation on the right-hand side of the model. We implement it by modifying (5) loss function as follows:

$$L = \frac{1}{2}\Psi(\mathbf{p}_1, \text{stopgrad}(z_2)) + \frac{1}{2}\Psi(\mathbf{p}_2, \text{stopgrad}(z_1)) \tag{7}$$

where stopgrad$(\cdot)$ denotes stop-gradient operation and this means that z_1 or z_2 is constant. The first term of the encoder on $\mathbf{x}^2$ receives no gradient from z_2, but the second term receives a gradient from $\mathbf{p}_2$ (as does $\mathbf{x}^1$).

C. Graph Convolutional Network

GCN [52] belongs to the class of graph neural network (GNN), which also uses convolutional operations. The graph in GCN is a complex nonlinear data structure, which is used to describe a one-to-many relationship in a non-Euclidean space. The graph is usually denoted by $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathbf{A})$, where $\mathcal{V}$ denotes the node set and $\mathcal{E}$ denotes the edge set. For each node $\mathbf{v}$, there is a feature vector $\mathbf{x}$. The node set can be represented as a matrix $\mathbf{X}^{n\times d}$, consisting of several feature vectors. n denotes the number of nodes, and d denotes the dimension of each feature vector. In addition, $\mathbf{A}$ denotes the adjacency matrix, which represents the relationships between nodes. Each element in $\mathbf{A}$ can be computed using the following

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 02:46:12 UTC from IEEE Xplore. Restrictions apply.

radial basis function (RBF):

$$\mathbf{A}_{i,j} = \exp\left(-\frac{\left\|\mathbf{s}_i - \mathbf{s}_j\right\|^2}{\varrho^2}\right) \tag{8}$$

where ϱ is a parameter to control the width of the RBF. Vectors $\mathbf{s}_i$ and $\mathbf{s}_j$ denote the spectral signatures associated with nodes $\mathbf{v}_i$ and $\mathbf{v}_j$, respectively.

As mentioned above, the GCN is also a CNN that performs layer-to-layer propagation in a way that

$$\mathbf{H}^{(l+1)} = \sigma\left(\tilde{\mathbf{D}}^{-\frac{1}{2}}\tilde{\mathbf{A}}\tilde{\mathbf{D}}^{-\frac{1}{2}}\mathbf{H}^{(l)}\mathbf{W}^{(l)}\right) \tag{9}$$

where $\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}$, $\mathbf{I}$ is the unit matrix. $\tilde{\mathbf{D}}$ is the degree matrix of $\tilde{\mathbf{A}}$, and $\tilde{\mathbf{D}}_{ii} = \sum_j \tilde{\mathbf{A}}_{ij}$. $\mathbf{H}$ is the characteristic of each layer, and for the input layer, $\mathbf{H}$ is $\mathbf{X}$. $\mathbf{W}^{(l)}$ is the weight parameter matrix for layer l. σ is a nonlinear activation function. The core of the GCN is based on the spectral decomposition of a symmetric normalized Laplacian matrix $\mathbf{L}^{\text{sym}}$ as follows:

$$\mathbf{L}^{\text{sym}} = \tilde{\mathbf{D}}^{-\frac{1}{2}}\hat{\mathbf{A}}\tilde{\mathbf{D}}^{-\frac{1}{2}} = \mathbf{I_n} - \tilde{\mathbf{D}}^{-\frac{1}{2}}\tilde{\mathbf{A}}\tilde{\mathbf{D}}^{-\frac{1}{2}} \tag{10}$$

where $\hat{\mathbf{A}} = \tilde{\mathbf{D}} - \tilde{\mathbf{A}}$. The normalization operation is performed on the adjacency matrix, obtained by multiplying both sides of the adjacency matrix by the degree of the node to open the square and then taking the inverse. Specifically, for each node pair $\mathbf{v}_i$ and $\mathbf{v}_j$, the elements of the matrix are calculated as (for undirected unweighted graphs)

$$\mathbf{L}^{\text{sym}}_{\mathbf{i,j}} := \begin{cases} 1, & \text{if } i = j \text{ and } \deg(\mathbf{v}_i) \neq 0 \\ -\dfrac{1}{\sqrt{\deg(\mathbf{v}_i)\deg(\mathbf{v}_j)}}, & \text{if } i \neq j \text{ and } \mathbf{v}_i \text{ is adj. to } \mathbf{v}_j \\ 0, & \text{otherwise} \end{cases} \tag{11}$$

where $\deg(\mathbf{v}_i), \deg(\mathbf{v}_j)$ is the value of the degree matrix at node $\mathbf{v}_i$ and node $\mathbf{v}_j$.

III. Methodology

In this section, we introduce our proposed S^{4}GPN in detail, and its framework is shown in Fig. 1. First, the framework starts with dual-path-based processing, including direct retention of the original spectral information and extended morphological profile (EMP) processing of spatial information. In addition to data processing, the framework consists of three stages, including the PN stage, the SSL stage, and the fusion stage. In the PN stage, the initial feature extractor f_θ which is made up of two branches, i.e., the GCN for extracting spectral information and the Densenet for extracting spatial information, is learned through the PN. After that, in the SSL stage, a self-supervised module augments the spectral and spatial data separately for SimSiam. The learned f_θ is used as a prior condition to optimize the feature extractor g_φ. Finally, in the fusion stage, the knowledge of $f_\theta(\mathbf{x})$ and $g_\varphi(\mathbf{x})$ that is learned in the first two stages is fused to optimize the final feature extraction network h_ω.

Algorithm 1 S^{4}GPN

Input: Input HSI;
Output: Predicted label for each pixel;
Process HSI using EMP algorithm;
Divide the support set S_i and the query set Q_i to construct the episode T_i;
1. Train f_θ in the PN stage;
for each episode T_i **do**
Obtain the extracted spectral features $\mathbf{H}_s^{(l)}$ and the extracted spatial features $\mathbf{H}_{\text{Dense}}^{(l)}$ according to (12) and (13);
Obtain the extracted features $\mathbf{H}_{\text{Fusion}}^{(l)}$ according to (14);
Compute the embedding center $\mathbf{c}_k$ of each class according to (1);
Calculate the distance $\mathbf{d}_k(\mathbf{x}^*)$ between each query sample $\mathbf{x}^*$ and $\mathbf{c}_k$ using (2);
Calculate the cross-entropy loss L_{PN} according to (4);
Update the parameters of feature extractor f_θ;
end for
2. Train g_φ in the SSL stage;
for each episode T_i **do**
Obtain the augmented spectral and spatial data $\mathbf{x}^1_{spe}$, $\mathbf{x}^2_{spe}$, $\mathbf{x}^1_{spa}$, $\mathbf{x}^2_{spa}$ separately in two different ways;
Obtain the extracted features $\mathbf{g}_1$, $\mathbf{g}_2$ using feature extractor g_φ with shared weights on two branches according to (12)–(14);
Calculate the symmetric loss L_{SSL} according to (7);
Obtain the extracted features $f_\theta(\mathbf{x})$, $g_\varphi(\mathbf{x})$ using feature extractors f_θ, g_φ;
Calculate the condition loss L_{CON} according to (16);
Add up the above-mentioned losses to obtain the total loss $L_{\text{T-SSL}}$ according to (17);
Update the parameters of feature extractor g_φ;
end for
3. Train h_ω in the fusion stage;
for each episode T_i **do**
Obtain the extracted features $f_\theta(\mathbf{x})$, $g_\varphi(\mathbf{x})$ using feature extractors f_θ, g_φ;
Obtain the feature matrix $\mathbf{Z}$ according to (18);
Obtain the similarity matrix $\mathbf{S}_{i,j}$ according to (19);
Obtain the weighted feature representation $h^*_\omega(\mathbf{x})$ according to (20) and (21);
Calculate the cross-entropy loss L_{FUS} according to (4);
Update the parameters of feature extractor h_ω;
end for
The final result is obtained by calculating the cross-entropy loss L_{FUS} according to (4);

A. PN Stage

The PN stage first trains the PN. For each episode T_i in the training process, we input the processed data from the data processing stage into the feature extractor f_θ to extract the corresponding feature vectors, which calculates the prototype features according to (1). Then the distances between the query features and the embedding center of each class are calculated and mapped. Finally, the cross-entropy loss function L_{PN} is optimized by SGD.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 02:46:12 UTC from IEEE Xplore. Restrictions apply.

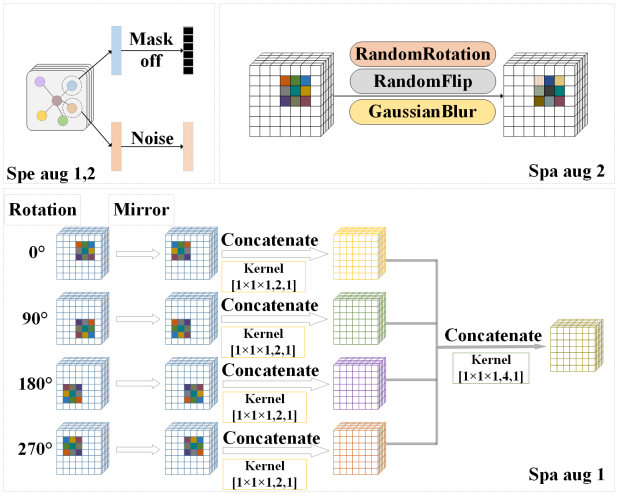


Fig. 2. Augmentation of the spectral and spatial data.

Throughout the framework, the feature extractors in each stage, including f_θ, g_φ, and h_ω, use the same network structure. As shown in Fig. 1, to make full use of the spectral and spatial information, the feature extraction network is composed of two branches. The first branch extracts spectral information by GCN, which consists of two graph convolution layers. The graph in GCN is constructed from the node set and the edge set. In the HIC, the node set consists of hyperspectral pixels, whereas the edge set is composed of the similarities between any two nodes, i.e., $\mathbf{v}_i$ and $\mathbf{v}_j$. Unlike CNN which uses minibatch as a training strategy, GCN usually only allows all the samples to be fed into the network at the same time for full batch network training. According to [36], the computational cost of GCN therefore becomes higher as the size of the graph increases. To address this issue, we instead apply the minibatch strategy to both CNN and GCN. Specifically, using (9), the GCN update rule in one batch is given by

$$\mathbf{H}_s^{(l+1)} = \sigma\left(\tilde{\mathbf{D}}_s^{-\frac{1}{2}}\tilde{\mathbf{A}}_s\tilde{\mathbf{D}}_s^{-\frac{1}{2}}\mathbf{H}_s^{(l)}\mathbf{W}_s^{(l)}\right) \tag{12}$$

where s refers to the minibatch in the network training, which can also be seen as all the samples in the support set S_i for episode T_i in FSL.

The second branch is for spatial feature extraction with Densenet

$$\mathbf{H}_{\text{Dense}}^{(l+1)} = \Phi^{(l+1)}\left(\left[\mathbf{H}_{\text{Dense}}^{(0)}, \mathbf{H}_{\text{Dense}}^{(1)}, \ldots, \mathbf{H}_{\text{Dense}}^{(l)}\right]\right), \quad l \in N^+ \tag{13}$$

where $\mathbf{H}_{\text{Dense}}^{(l+1)}$ is the output of the $(l+1)$th layer and $\Phi^{(l+1)}$ represents the complex nonlinear transformation operation in the $(l+1)$th convolution layer. The feature maps of the output of the 0th to the lth layers act as the input to the $(l+1)$th layer.

Densenet usually consists of multiple densely connected blocks. As shown in Fig. 1, in this article, the spatial feature extraction processing consists of a convolutional layer, a dense spatial block, a 2-D average pooling layer, and a 2-D adaptive max-pooling layer. The dense spatial block contains three 2-D convolutional layers with a convolutional kernel size of 3×3.

After separate feature extraction to obtain spectral and spatial feature representations, they are fused as follows:

$$\mathbf{H}_{\text{Fusion}}^{l} = \left[\mathbf{H}_s^{(l)}, \mathbf{H}_{\text{Dense}}^{(l)}\right] \tag{14}$$

where the operator $[\cdot,\cdot]$ denotes the elementwise concatenation. $\mathbf{H}_s^{(l)}$ and $\mathbf{H}_{\text{Dense}}^{(l)}$ represent the lth layer features extracted by GCN and Densenet, respectively.

B. SSL Stage

In the SSL stage, we use a SimSiam structure to implement SSL to provide additional semantic information for FSL. For each episode T_i, we input the augmented data from the support set and query set into the SimSiam structure. Types of data augmentation are shown in Fig. 2.

Different data augmentation methods are used for the spectral and spatial data. For the two branches of the SimSiam, we augment the spectral data with random masking and random noise addition to obtain the augmented data $\mathbf{x}_{\text{spe}}^1$ and $\mathbf{x}_{\text{spe}}^2$. In the spectral feature extraction process, the spectral data of the images form an undirected graph as nodes. Therefore, the augmentation of the spectral data is to augment the node features of the graph. Random masking is achieved by randomly removing node features and their connections to all the neighbors. Random noise addition is achieved by randomly adding Gaussian noise to node features. Gaussian noise is a noise generated by adding to the input data normally distributed random values with a mean of 0 and a standard deviation of 0.5. Spatial data are also augmented in two different ways, one of which is through rotation plus mirroring. Due to the rotational invariance and spatial symmetry of HSI, the horizontal rotation of the image does not affect the class prediction of HSI pixels. We rotate the image horizontally in the spatial domain by four angles, the corresponding set of rotation angles is $T = \{90^\circ \cdot r | r \in [0, 1, 2, 3]\}$, and the resulting rotated image is $\{\mathbf{H}_t | t \in T\}$, where $\mathbf{H}_t$ is the replica of image $\mathbf{H}$ rotated by t degrees. Subsequently, the image is mirrored in the spatial dimension to obtain the mirrored image $\overline{\mathbf{H}}_t$. The rotated image is concatenated with the mirrored image to obtain an image matrix of size $W \times H \times D \times 2$. We perform a 3-D convolution operation on the resulting image as follows:

$$\mathbf{H}_t^* = \text{Relu}(\text{BN}(\text{Conv}([\mathbf{H}_t, \overline{\mathbf{H}}_t]))) \tag{15}$$

where the size of the convolution kernel for 3-D convolution is $1 \times 1 \times 2$, and $\mathbf{H}_t^*$ denotes the image matrix after the convolution operation.

We perform the above operations for each of the four rotation angles and then perform the concatenation and 3-D convolution operations again to obtain the augmented data $\mathbf{x}_{\text{spa}}^1$ to efficiently extract the geometrically transformed features. Another branch of spatial data augmentation is the traditional random rotation, random flipping, and random addition of Gaussian noise to obtain the augmented data $\mathbf{x}_{\text{spa}}^2$.

After obtaining the augmented data $\mathbf{x}_{\text{spe}}^1$, $\mathbf{x}_{\text{spe}}^2$, $\mathbf{x}_{\text{spa}}^1$, and $\mathbf{x}_{\text{spa}}^2$, we feed the two sets of spectral and spatial augmented

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 02:46:12 UTC from IEEE Xplore. Restrictions apply.

TABLE I
LAND-COVER CLASSES AND SAMPLES FOR THE PU DATASET

#	Class	Samples
1	Asphalt	6631
2	Meadows	18649
3	Gravel	2099
4	Trees	3064
5	Painted metal sheets	1345
6	Bare Soil	5029
7	Bitumen	1330
8	Self-Blocking Bricks	3682
9	Shadows	947
-	Total	42776

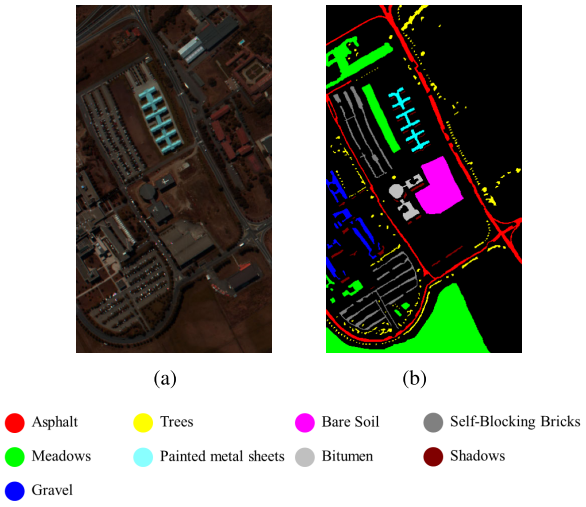


Fig. 3. PU dataset. (a) False-color image. (b) Ground-truth map.

data separately into an encoder network for processing, which consists of the feature extractor g_φ and the projection MLP head ζ, and obtain the feature representations $\mathbf{g}_1$ and $\mathbf{g}_2$. The symmetric loss L_{SSL} is calculated using (5) between the feature representations $\mathbf{g}_1$, $\mathbf{g}_2$.

To better integrate SSL with FSL, we optimize the feature manifold learned in the SSL stage using the feature learned in the PN stage as a prior condition. Specifically, we minimize the negative cosine similarity between the feature representations extracted by the feature extractor f_θ and g_φ, as follows:

$$L_{\text{CON}}(\mathbf{x}) = -\frac{f_\theta(\mathbf{x})}{\|f_\theta(\mathbf{x})\|_2} \cdot \frac{g_\varphi(\mathbf{x})}{\|g_\varphi(\mathbf{x})\|_2}. \tag{16}$$

Then, the condition loss is combined with the self-supervised loss to obtain the total loss function for the SSL stage as follows:

$$L_{\text{T-SSL}}(\mathbf{x}) = \mathop{\mathbb{E}}_{\mathbf{x}\in\mathcal{D}_{tr}} \left[L_{\text{SSL}}(\mathbf{x}) + \gamma L_{\text{CON}}(\mathbf{x})\right] \tag{17}$$

where γ is a constant that weighs the importance of self-supervised loss and condition loss.

C. Fusion Stage

To further improve the capability of the feature representation, we fuse the features learned in the first two stages. Specifically, for sample $\mathbf{x}_i$, the two corresponding embedding vectors $f_\theta(\mathbf{x}_i)$ and $g_\varphi(\mathbf{x}_i)$ are calculated in each episode T_i,

TABLE II
LAND-COVER CLASSES AND SAMPLES FOR THE IP DATASET

#	Class	Samples
1	Alfalfa	46
2	Corn-notill	1428
3	Corn-mintill	830
4	Corn	237
5	Grass-pasture	483
6	Grass-trees	730
7	Grass-pasture-mowed	28
8	Hay-windrowed	478
9	Oats	20
10	Soybean-notill	972
11	Soybean-mintill	2455
12	Soybean-clean	593
13	Wheat	205
14	Woods	1265
15	Buildings-Grass-Trees-Drives	386
16	Stone-Steel-Towers	93
-	Total	10249

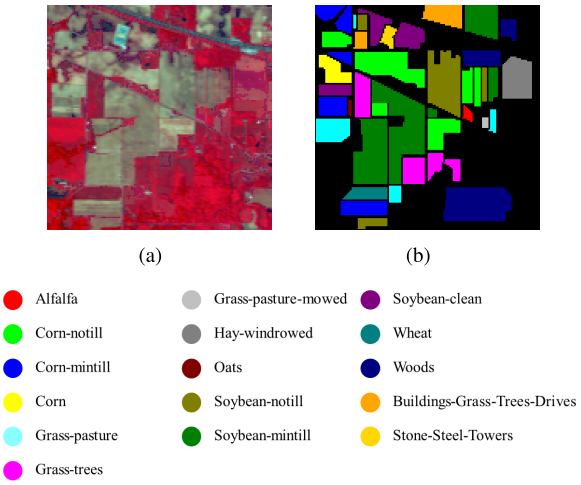


Fig. 4. IP dataset. (a) False-color image. (b) Ground-truth map.

TABLE III
LAND-COVER CLASSES AND SAMPLES FOR THE HOUSTON DATASET

#	Class	Samples
1	Healthy grass	1251
2	Stressed grass	1254
3	Synthetic grass	697
4	Trees	1244
5	Soil	1242
6	Water	325
7	Residential	1268
8	Commercial	1244
9	Road	1252
10	Highway	1227
11	Railway	1235
12	Parking Lot 1	1233
13	Parking Lot 2	469
14	Tennis Court	428
15	Running Track	660
-	Total	15029

and then the two are concatenated to achieve augmentation as follows:

$$\mathbf{z}_i = C(f_\theta(\mathbf{x}_i), g_\varphi(\mathbf{x}_i)) \tag{18}$$

where $C(\cdot,\cdot)$ denotes concatenation. By calculating $\mathbf{z}_i$ for different samples, the feature matrix $\mathbf{Z} = [\mathbf{z}_1, \mathbf{z}_2, \ldots, \mathbf{z}_n]^T$ corresponding to these samples can be obtained, where n

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 02:46:12 UTC from IEEE Xplore. Restrictions apply.

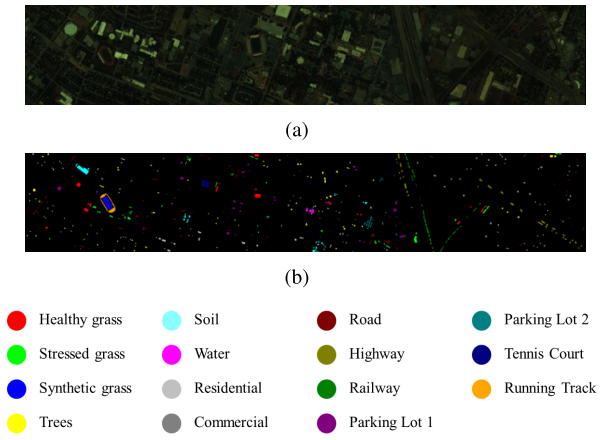


Fig. 5. Houston dataset. (a) False-color image. (b) Ground-truth map.

TABLE IV
LAND-COVER CLASSES AND SAMPLES FOR THE SALINAS DATASET

#	Class	Samples
1	Brocoli-green-weeds-1	2009
2	Brocoli-green-weeds-2	3726
3	Fallow	1976
4	Fallow-rough-plow	1394
5	Fallow-smooth	2678
6	Stubble	3959
7	Celery	3579
8	Grapes-untrained	11271
9	Soil-vinyard-develop	6203
10	Corn-senesced-green-weeds	3278
11	Lettuce-romaine-4wk	1068
12	Lettuce-romaine-5wk	1927
13	Lettuce-romaine-6wk	916
14	Lettuce-romaine-7wk	1070
15	Vinyard-untrained	7268
16	Vinyard-vertical-trellis	1807
-	Total	54129

is the number of samples. To further extract the feature representation, we perform a 1-D convolution operation on the obtained $\mathbf{Z}$ with a kernel size of 1 to obtain $\mathbf{Z}^*$. After regularizing $\mathbf{Z}^*$, we calculate the cosine similarity between the two samples as follows:

$$\mathbf{S}_{i,j} = \begin{cases} \dfrac{\mathbf{Z}^*_{i,:}(\mathbf{Z}^*_{j,:})^T}{\|\mathbf{Z}^*_{i,:}\|_2 \|\mathbf{Z}^*_{j,:}\|_2}, & i \neq j \end{cases} \tag{19}$$

where $\mathbf{S}_{i,j}$ denotes the feature similarity between the ith and jth sample. Next, we normalize the similarity matrix to highlight commonalities as follows:

$$\delta = \frac{\exp(\mathbf{S}_{i,j})}{\sum_{k=1}^{n} \exp(\mathbf{S}_{i,k})}. \tag{20}$$

After that, we weight the obtained δ onto the feature representation of h_ω as follows:

$$h^*_\omega(\mathbf{x}) = \delta \cdot h_\omega(\mathbf{x}) + h_\omega(\mathbf{x}). \tag{21}$$

Finally, similar to the PN stage, the embedding centers are calculated based on the weighted feature representation $h^*_\omega(\mathbf{x})$ and the feature representation $h_\omega(\mathbf{x})$ of the support set, respectively. The similarity between the query set sample and the embedding center is calculated, and then the cross-entropy loss L_{FUS} is calculated according to (4). Algorithm 1 summarizes our proposed S^{4}GPN method.

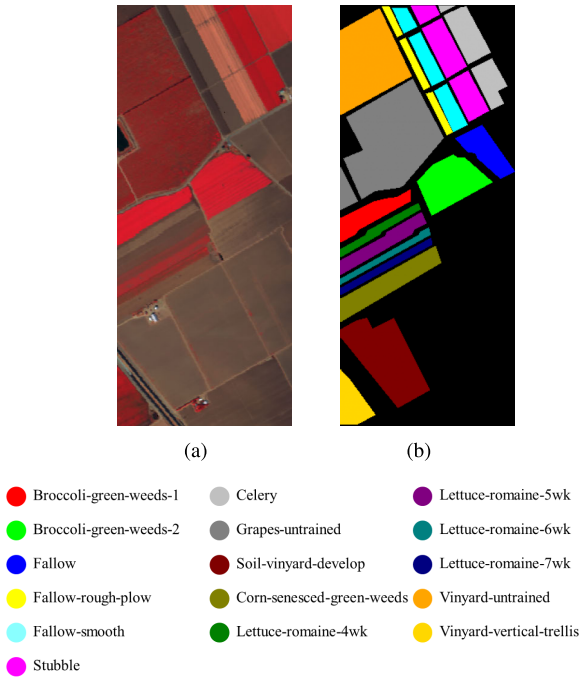


Fig. 6. Salinas dataset. (a) False-color image. (b) Ground-truth map.

IV. EXPERIMENTS

In this section, we experiment with our S^{4}GPN on four public HSI datasets, including the Pavia University (PU) dataset, the Indian Pines (IP) dataset, the Salinas dataset, and the Houston dataset. The performance of the experiment is evaluated by three metrics, including overall accuracy (OA), average accuracy (AA), and kappa coefficient (Kappa).

A. HSI Datasets

1) The PU dataset is an image acquired during a flight over Pavia in Northern Italy, taken by the ROSIS sensor, with a spectral range of 430–860 nm in 115 bands and a spatial resolution of 1.3 m. In all, 103 bands are actually used for training, with a size of 610 × 340, for nine classes of features. Table I shows the land-cover classes and the number of samples of the PU dataset. Fig. 3 shows the labeled samples.
2) The IP dataset was imaged in 1992 in IN, USA, and cropped to 145 × 145 for HIC. The imager was an airborne visible/infrared imaging spectrometer (AVIRIS), which has a wavelength range of 400–2500 nm. The dataset has a spatial resolution of approximately 20 m and contains 220 continuous bands. After removing 20 absorption bands (104–105, 150–163, 220), the actual bands used for training are 200. There are 16 classes in the study area. Table II shows the land-cover classes and the number of samples of the IP dataset. Fig. 4 shows the false-color image and ground-truth map of the IP dataset.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 02:46:12 UTC from IEEE Xplore. Restrictions apply.

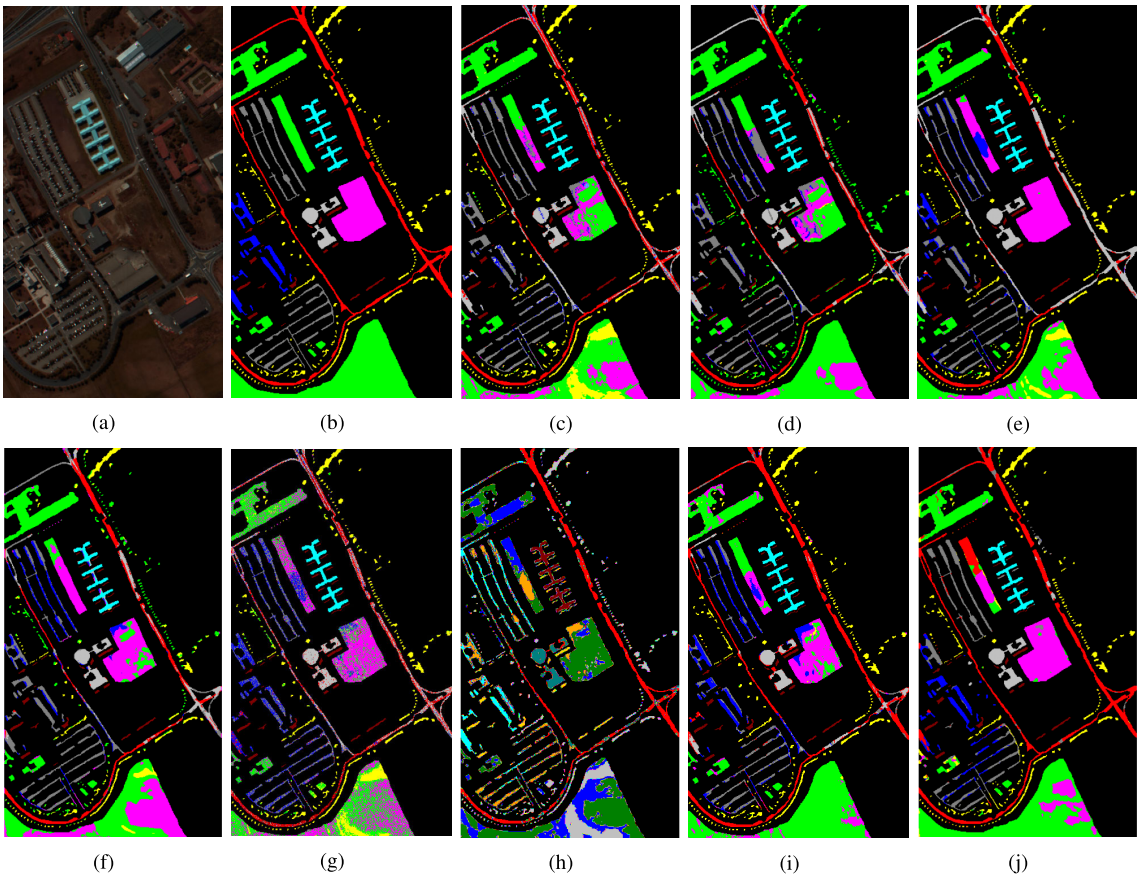


Fig. 7. Illustration of classification results on the PU dataset. (a) False-color image. (b) Ground truth. (c) SVM. (d) 3D-CNN. (e) SDRN. (f) FDSSC. (g) GCN. (h) DFSL. (i) DCFSL. (j) S^{4}GPN.

TABLE V

OA (%) OF THE PROPOSED S^{4}GPN METHOD WITH DIFFERENT INPUT SIZES $c \times c$

$c \times c$	PU	IP	Salinas	Houston
3×3	79.36±5.29	59.37±4.07	88.03±3.00	71.85±1.71
5×5	86.03±4.14	67.33±3.11	87.67±3.18	76.58±1.95
7×7	85.83±3.25	70.52±3.43	90.22±3.03	81.02±1.54
9×9	**86.34±3.38**	73.00±3.17	**90.94±3.53**	**81.32±1.51**
11×11	86.33±5.21	**73.86±2.60**	90.08±3.17	76.42±2.05

3) The Houston dataset was acquired by the ITERS CASI-1500 sensor in Houston, TX, USA, and its surrounding rural areas at a spatial resolution of 2.5 m. The data size is 349 × 1905 and contains 144 bands in the wavelength range of 364–1046 nm. Fifteen feature types are present in the study area, including roads, land, trees, highways, etc. Table III shows the land-cover classes and the number of samples of the Houston dataset. There are 15 classes in the image as displayed in Fig. 5.
4) The Salinas dataset was taken by the AVIRIS sensor in Salinas Valley, CA, USA. The dataset is 512 × 217 in size, has a spatial resolution of 3.7 m, and contains 224 continuous bands. After removing the 20 vapor absorption bands (108–112, 154–167, 224), the actual bands used for training are 204. There are 16 classes in the study area. Table IV shows the land-cover classes and the number of samples of the Salinas dataset. False-color image and ground-truth map of the Salinas dataset for 16 classes are shown in Fig. 6.

TABLE VI

OA (%) OF THE PROPOSED S^{4}GPN METHOD WITH DIFFERENT HIGHEST ORDER OF NEIGHBORS r

r	PU	IP	Salinas	Houston
1	85.99±3.07	72.34±2.60	90.14±3.90	80.47±2.85
2	84.73±5.17	72.68±3.14	90.12±3.27	80.98±2.28
3	**86.51±3.88**	72.38±3.42	90.53±3.50	78.40±2.22
4	85.49±5.40	73.63±2.41	90.61±3.88	80.47±1.62
5	85.71±4.61	73.32±2.84	90.56±2.03	77.78±2.44
6	85.17±3.73	72.27±2.43	90.01±3.15	80.41±2.45
7	85.45±4.72	73.24±3.35	90.75±2.94	77.97±2.64
8	86.34±3.38	73.00±3.17	**90.94±3.53**	**81.32±1.51**
9	85.31±3.94	**74.11±1.73**	90.55±3.64	77.56±3.60
10	86.30±4.51	72.52±1.84	90.54±3.53	80.41±1.54

B. Experimental Settings

In the training process, we set the epoch numbers for the PN and fusion stages to 200 and for the SSL stage to 20 to allow the network to reach a certain level of convergence, and the learning rate for all the three key stages is set to 0.001. We use Adam optimizer to optimize the parameters. In each epoch, we set the number of episodes to 100, and each episode is a K-shot, C-way classification task. C is set to the class number in each episode (e.g., PU dataset is 9, Houston dataset is 15, IP and Salinas datasets are 16), and K is the number of samples per class in the support sets. In our experiment, K is set to 1. By experimental verification, we set the input patch size of the network as 9 × 9. The experiments have been run 10× with different initial training samples to eliminate the influence of random sampling. All the experiments are

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 02:46:12 UTC from IEEE Xplore. Restrictions apply.

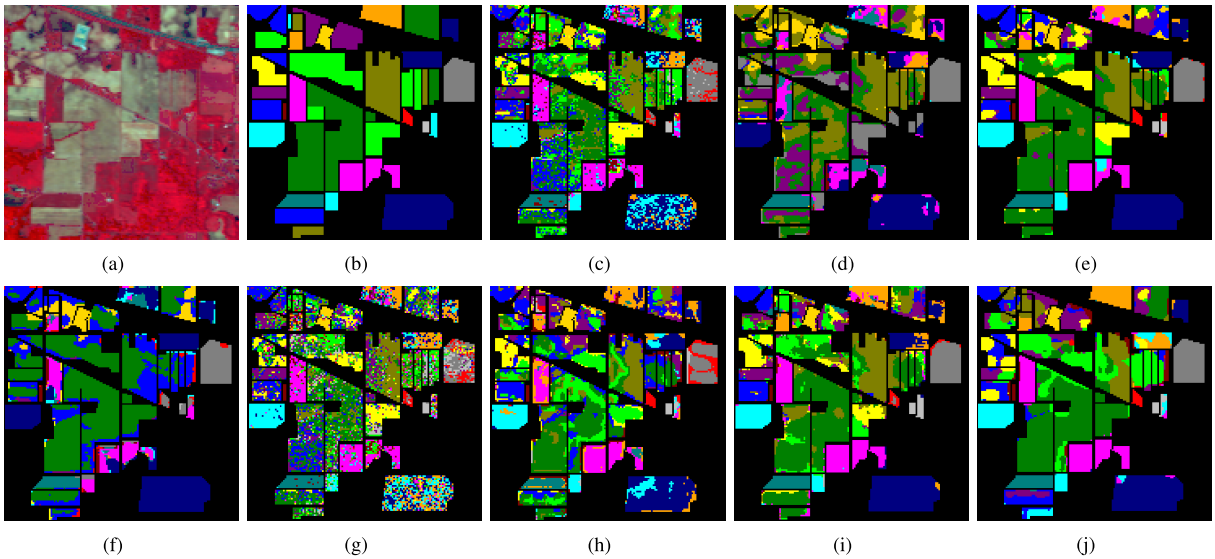


Fig. 8. Illustration of classification results on the IP dataset. (a) False-color image. (b) Ground truth. (c) SVM. (d) 3D-CNN. (e) SDRN. (f) FDSSC. (g) GCN. (h) DFSL. (i) DCFSL. (j) S^{4}GPN.

TABLE VII
CLASSIFICATION RESULTS ON THE PU DATASET (FIVE LABELED SAMPLES PER CLASS)

Class	SVM	3D-CNN	SDRN	FDSSC	GCN	DFSL	DCFSL	S^{4}GPN
1	51.15±17.84	53.04±11.58	80.53±5.85	50.92±18.41	47.96±25.58	74.14±11.25	79.09±12.93	**94.01±4.37**
2	53.54±21.46	55.73±11.56	70.09±12.60	66.34±16.27	57.76±14.54	58.18±22.53	**84.94±8.19**	81.94±6.47
3	26.56±17.80	45.75±16.45	80.42±11.21	75.60±10.50	65.51±11.35	70.44±14.55	62.68±10.31	**82.66±9.46**
4	90.21±8.65	86.89±9.86	91.45±5.16	89.04±6.81	79.40±15.21	86.79±5.06	92.40±4.92	**93.62±3.17**
5	98.54±1.78	89.56±5.36	**99.96±0.08**	99.03±2.16	95.72±5.25	99.31±1.09	98.38±1.95	99.81±0.40
6	50.88±22.20	45.51±13.57	84.50±12.27	85.20±12.08	68.08±17.84	76.04±9.55	73.63±6.97	**87.55±9.89**
7	89.96±7.67	83.43±6.66	95.06±9.06	87.77±10.95	72.41±14.24	87.01±5.77	78.58±6.09	**99.05±1.12**
8	68.81±20.63	64.42±20.76	75.98±11.06	49.95±18.76	63.29±16.51	53.56±20.52	61.67±12.84	**78.87±16.80**
9	99.83±0.14	99.16±0.68	**99.85±0.29**	99.84±0.19	91.43±16.43	98.30±0.86	98.37±1.82	90.26±6.80
OA(%)	59.03±6.53	59.47±5.48	78.31±6.22	69.25±7.40	63.93±5.44	68.09±11.42	80.67±3.32	**86.34±3.38**
AA(%)	69.95±2.22	69.28±2.11	86.43±2.83	78.79±2.45	72.07±4.64	78.20±3.84	81.08±1.66	**89.75±2.06**
Kappa(×100)	49.90±5.82	50.52±5.16	72.93±7.11	62.01±8.00	55.49±5.44	61.28±12.00	74.99±3.87	**82.47±4.17**

run on a server with an NVIDIA GTX 3060 GPU using PyTorch.

C. Parameters' Settings

In this section, we analyze the impact of two hyperparameters, the input size of the data, and the highest order of the neighbors in the graph convolution. We compare the classification performances of our S^{4}GPN on four datasets by different parameter settings.

1) Spatial Size $c \times c$: Our experiment extracts the spectral and spatial information separately, so the data preprocessing stage processes the data in two paths into spatial data of size $c \times c$ and spectral data of size B and then inputs them into the network. We calculate the OA using different spatial sizes with values in the range [3, 5, 7, 9, 11]. From Table V, it can be observed that except for the IP dataset, OA shows a tendency to increase and then decrease with increasing spatial size, yielding a better OA at the size of 9×9. Moreover, as the spatial size increases, the training time of the network also increases. Therefore, considering the need for both classification performance and computational efficiency, we set the spatial size as 9×9 in all the experiments.

2) Highest Order of the Neighbors r: In our method, r is the highest order neighbor used for feature learning in graph convolution processing where spectral feature extraction is performed. To find the optimal r, we set r from 1 to 10 and conduct experiments. Table VI shows the experimental results and it can be seen that the OA is higher on the PU and Salinas datasets when r is taken as 8, on the IP dataset when r is taken as 9, and on the PU dataset when r is taken as 3. As r becomes larger, the information learned is probably redundant, which is detrimental to feature learning and leads to the degradation of classification performance. Accordingly, in our experiments, r takes the value of 8.

D. Experimental Results

To verify the effectiveness of the S^{4}GPN, we compare our method with seven methods, including SVM [12], 3D-CNN [25], SDRN [28], FDSSC [31], GCN [52], DFSL [39], and DCFSL [42]. These include five supervised learning methods, i.e., SVM, 3D-CNN, SDRN, FDSSC, and GCN. The SVM uses kernel methods to map nonlinear data to a linearly separable high-dimensional feature space. This method only extracts the 1-D spectral information and ignores the 2-D spatial information. The 3D-CNN can effectively extract deep joint spectral–spatial features from HSI cube data.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 02:46:12 UTC from IEEE Xplore. Restrictions apply.

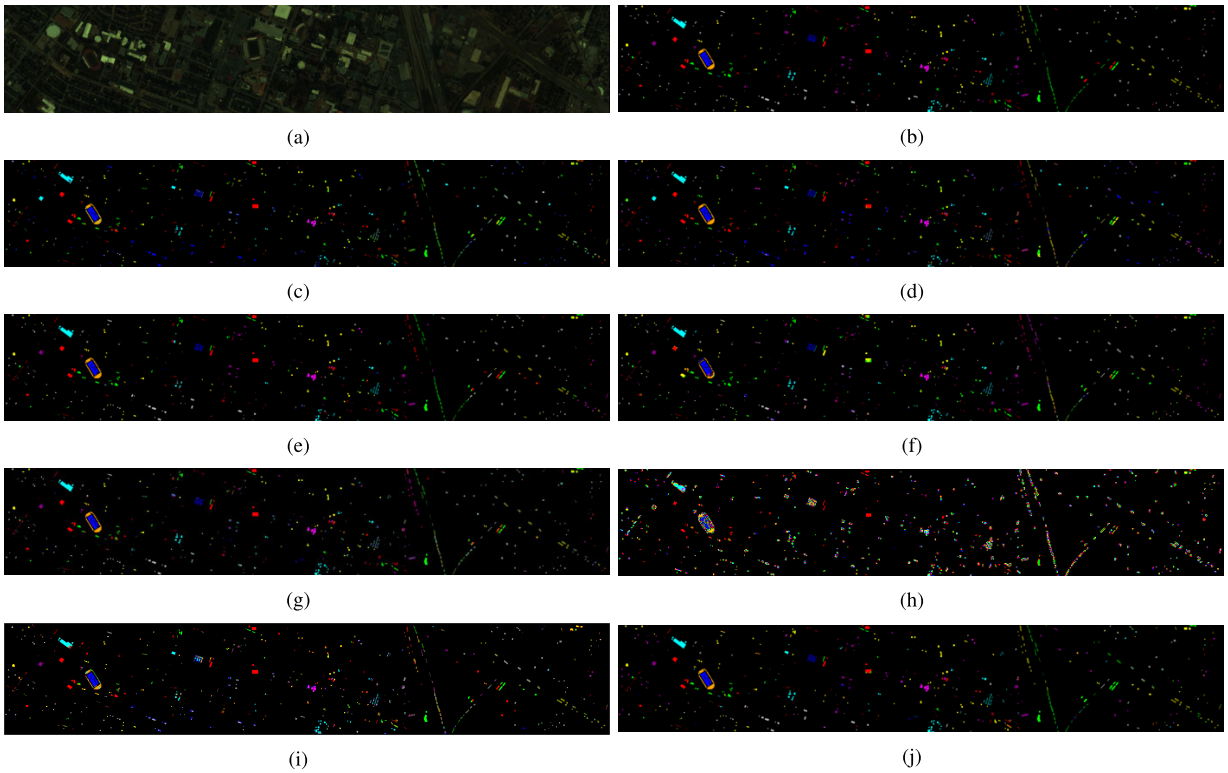


Fig. 9. Illustration of classification results on the Houston dataset. (a) False-color image. (b) Ground truth. (c) SVM. (d) 3D-CNN. (e) SDRN. (f) FDSSC. (g) GCN. (h) DFSL. (i) DCFSL. (j) S^{4}GPN.

TABLE VIII
CLASSIFICATION RESULTS ON THE IP DATASET (FIVE LABELED SAMPLES PER CLASS)

Class	SVM	3D-CNN	SDRN	FDSSC	GCN	DFSL	DCFSL	S^{4}GPN
1	75.12±14.17	81.67±19.05	95.28±6.09	76.11±16.81	49.76±27.43	81.96±13.01	92.68±9.45	**98.33±3.56**
2	32.28±6.66	35.51±20.11	50.41±6.63	24.68±12.79	33.32±9.84	35.41±9.36	49.16±10.47	**55.16±9.83**
3	41.68±6.01	21.65±7.76	53.13±5.58	29.96±14.15	36.65±9.57	51.76±10.90	51.31±7.57	**70.37±12.25**
4	49.61±7.26	36.04±14.36	73.48±15.90	40.26±10.39	35.46±10.16	73.00±11.26	79.14±9.33	**82.51±14.83**
5	59.56±19.69	45.56±20.87	78.71±8.98	65.39±9.11	53.12±17.38	70.93±17.13	72.49±8.11	**82.18±6.43**
6	64.41±10.25	68.07±13.22	**95.01±5.34**	83.81±9.75	67.29±11.04	83.26±6.07	84.32±8.22	87.68±7.72
7	84.78±11.38	87.78±8.89	**100.00±0.00**	92.78±8.62	85.94±7.51	99.29±1.43	99.57±1.30	**100.00±0.00**
8	64.02±17.62	72.33±22.93	90.36±7.31	68.46±14.86	54.54±22.33	72.26±9.06	85.58±11.74	**99.42±0.71**
9	84.67±15.22	**100.00±0.00**	**100.00±0.00**	99.00±3.00	77.09±21.83	97.50±3.35	99.33±2.00	**100.00±0.00**
10	38.97±10.32	45.94±7.98	58.56±11.14	52.91±15.62	50.26±26.07	44.90±7.65	61.02±7.55	**70.96±9.07**
11	41.58±11.87	38.50±17.99	46.96±15.44	44.08±17.00	30.51±10.05	42.04±14.03	59.97±10.20	**63.12±12.72**
12	29.30±11.06	27.86±8.86	**65.49±15.90**	27.60±7.56	30.34±3.06	27.52±11.34	43.74±9.86	57.74±7.77
13	89.50±5.14	91.28±7.83	**99.28±0.92**	96.05±7.66	77.81±27.92	89.27±5.54	98.15±3.34	98.56±2.65
14	67.03±16.95	62.57±16.31	84.42±9.34	79.69±9.67	60.00±18.40	81.11±5.19	83.54±7.82	**86.06±8.87**
15	21.99±9.56	39.20±14.87	76.12±13.09	49.26±16.62	35.90±9.77	58.76±6.58	70.79±9.84	**94.23±5.36**
16	88.41±5.99	87.11±5.70	**100.00±0.00**	96.51±6.59	76.06±23.11	95.59±5.14	98.75±2.66	97.47±5.48
OA(%)	47.09±1.95	45.52±5.17	65.17±3.24	51.29±2.11	40.16±2.17	55.09±3.42	65.56±3.74	**73.00±3.17**
AA(%)	58.31±1.08	58.82±3.01	79.20±1.11	64.16±2.21	53.81±2.26	69.03±2.17	76.85±1.98	**83.99±1.82**
Kappa(×100)	40.80±1.91	38.91±5.37	60.96±3.34	45.48±2.20	34.10±2.14	49.83±3.56	61.22±4.08	**69.67±3.36**

The GCN method solves the problem of missing class boundary information by extracting features through graph structure, but similar to the SVM method, only 1-D spectral information is extracted. The DFSL can learn the metric space through a deep residual 3D-CNN structure and classify samples through the NN classifier. The DCFSL achieves cross-domain HIC through an adversarial domain adaptive strategy. In the DCFSL method, according to [42], we set the Chikusei dataset as the source domain data and select 200 labeled samples per class randomly. To ensure the fairness of the experiments, we take two to ten labeled samples from each of the four datasets for training in all the compared experiments. According to [36], the graph constructed by taking only one labeled sample per class cannot provide non-Euclidean irregular data to mitigate the boundary information loss problem when using GCN to extract spectral features. Therefore, one labeled sample per class is not taken for the experiment. In the GCN and S^{4}GPN, the spectral data are extracted using the GCN structure, and the graph constructed in GCN requires at least two samples. For all the compared experiments, we select five labeled samples in each class for training.

1) Results on the PU Dataset: The experimental results of our S^{4}GPN and the compared methods on the PU dataset are listed in Table VII, with the optimal classification results in bold. Our S^{4}GPN has a great advantage in all three metrics. The OA, AA, and Kappa of our method improve by 5.67%, 8.67%, and 0.0748, respectively, in comparison

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 02:46:12 UTC from IEEE Xplore. Restrictions apply.

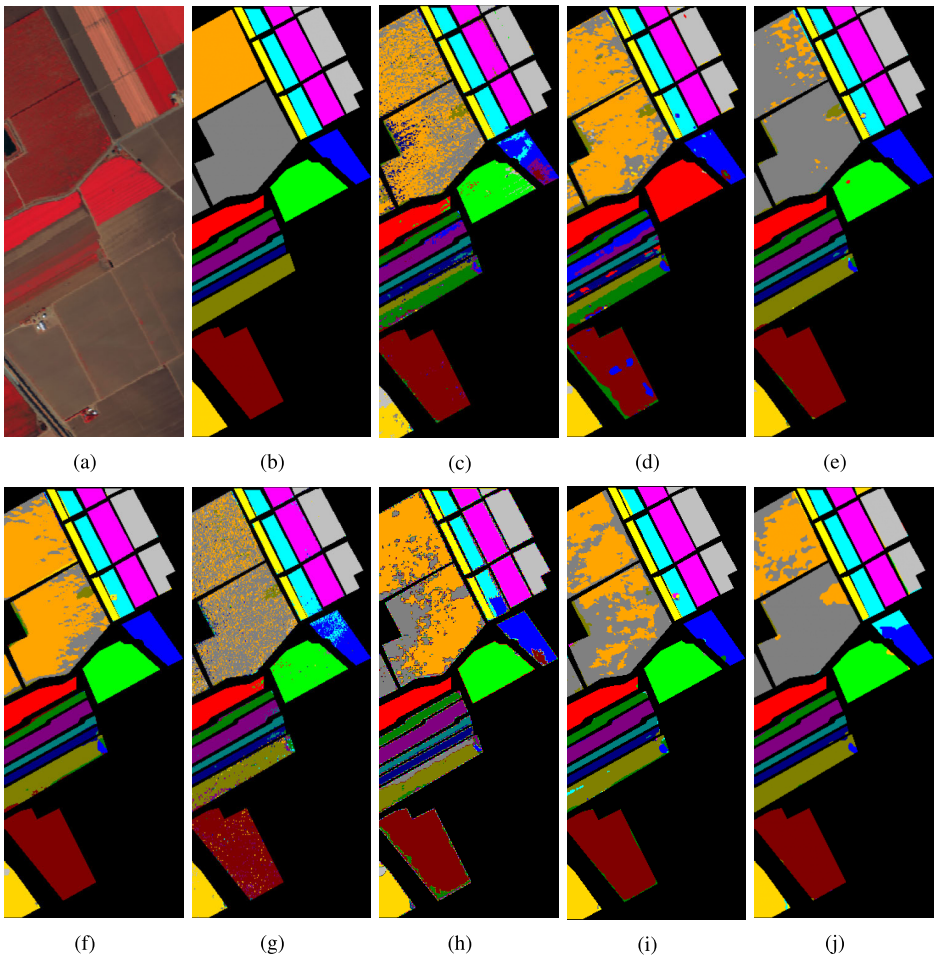


Fig. 10. Illustration of classification results on the Salinas dataset. (a) False-color image. (b) Ground truth. (c) SVM. (d) 3D-CNN. (e) SDRN. (f) FDSSC. (g) GCN. (h) DFSL. (i) DCFSL. (j) S^{4}GPN.

TABLE IX

CLASSIFICATION RESULTS ON THE HOUSTON DATASET (FIVE LABELED SAMPLES PER CLASS)

Class	SVM	3D-CNN	SDRN	FDSSC	GCN	DFSL	DCFSL	S^{4}GPN
1	85.28±1.65	84.54±4.93	90.88±4.85	**92.39±5.55**	79.75±26.88	89.64±5.11	89.28±5.29	91.71±5.40
2	59.59±14.43	56.11±18.71	89.63±6.48	85.92±14.11	87.86±6.73	94.21±4.47	**97.34±1.91**	94.91±4.29
3	97.96±0.62	90.28±6.66	**99.80±0.12**	98.72±1.04	98.25±1.70	96.00±2.90	97.34±1.91	98.54±1.03
4	87.38±8.16	80.39±9.53	91.73±5.91	94.42±3.25	91.93±2.77	95.80±2.64	94.83±3.22	**96.27±2.38**
5	71.63±11.56	82.22±9.05	97.69±3.85	97.62±2.04	95.01±3.97	92.62±6.47	95.44±3.11	**98.30±2.92**
6	71.87±0.42	62.76±15.64	**89.65±7.28**	82.41±4.88	79.34±7.38	74.77±7.98	83.16±7.95	88.19±5.87
7	28.70±15.75	44.78±13.80	70.17±11.07	64.71±14.69	68.76±7.46	77.20±8.61	**83.27±6.75**	80.65±10.08
8	33.63±8.76	49.88±7.24	51.63±11.45	52.05±6.37	38.95±13.02	**59.90±12.29**	49.12±4.23	48.48±14.65
9	63.15±12.72	40.05±13.52	64.29±11.79	58.69±15.92	53.65±12.88	**72.16±4.23**	67.43±11.24	68.62±9.49
10	14.43±13.77	55.28±19.66	62.94±14.46	54.00±14.25	48.29±14.21	52.27±12.97	69.35±11.24	**73.34±13.07**
11	53.12±20.95	44.27±10.15	71.00±15.59	63.64±11.88	54.14±8.41	41.37±24.11	65.80±5.15	**71.58±13.17**
12	3.83±7.07	34.78±10.18	50.69±15.06	54.67±17.06	40.39±7.19	65.61±18.03	**70.84±11.30**	63.21±14.71
13	5.34±10.40	48.50±16.52	86.47±7.76	57.32±12.62	36.96±7.96	88.04±2.77	**89.85±5.52**	81.26±6.58
14	89.88±5.36	76.63±10.86	**99.28±1.09**	99.23±0.99	86.91±17.28	95.28±3.80	92.67±5.72	96.58±2.54
15	92.92±2.92	90.20±4.98	**99.18±0.98**	98.65±1.94	96.58±0.81	96.44±3.10	95.68±3.37	98.95±0.95
OA(%)	54.44±1.98	60.59±2.82	78.65±1.72	74.81±3.10	69.34±2.05	77.20±2.97	79.31±2.21	**81.32±1.51**
AA(%)	57.25±1.51	62.71±2.15	81.67±1.53	76.96±2.58	70.65±2.46	79.42±2.42	79.31±2.21	**83.37±1.15**
Kappa(×100)	50.90±2.07	57.44±3.03	76.95±1.86	72.78±3.33	66.89±2.20	75.35±3.23	77.66±2.38	**79.81±1.62**

to the second best performing FSL model DCFSL, which fully demonstrates that compared with using only FSL, the fusion of SSL and FSL can further improve the quality of the feature representation based on limited labeled samples. Fig. 7 shows the visual classification results for the different methods and it can be observed that our method outperforms the others in terms of the smoothness of the boundaries.

2) Results on the IP Dataset: The results of experiments with various methods for the IP dataset are listed in Table VIII, where bold indicates the best classification results. From Table VIII, it can be seen that the classification accuracy of our method is optimal for most classes (Alfalfa, Corn-notill, Corn-mintill, Corn, Grass-pasture, etc.), indicating that the staged method of optimizing the feature extractor is beneficial to improve the classification accuracy of each class in a

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 02:46:12 UTC from IEEE Xplore. Restrictions apply.

TABLE X
CLASSIFICATION RESULTS ON THE SALINAS DATASET (FIVE LABELED SAMPLES PER CLASS)

Class	SVM	3D-CNN	SDRN	FDSSC	GCN	DFSL	DCFSL	S^{4}GPN
1	97.72±0.91	79.77±14.03	98.59±3.15	97.28±4.11	78.56±19.29	**99.21±0.75**	98.77±2.56	99.02±2.71
2	88.85±8.52	86.39±8.48	99.98±0.06	**100.00±0.00**	96.51±4.87	98.12±1.70	99.35±0.71	99.74±0.42
3	62.01±15.12	81.92±14.43	93.90±8.93	90.32±8.71	89.30±5.98	91.28±4.95	**94.41±7.20**	85.94±10.92
4	98.89±0.60	98.02±3.29	**99.89±0.09**	99.23±0.97	94.40±8.32	96.45±3.92	99.60±0.46	99.01±1.20
5	93.82±5.84	88.82±8.59	95.93±2.51	93.64±6.75	**96.47±2.45**	87.23±6.26	91.77±2.17	95.74±2.32
6	97.61±1.33	98.74±0.91	**99.89±0.28**	99.81±0.19	98.97±0.68	96.12±1.94	99.44±0.51	99.79±0.38
7	98.97±0.78	94.84±11.90	**99.98±0.02**	99.84±0.13	99.33±0.82	96.07±3.12	98.31±4.01	96.98±2.91
8	50.99±19.97	54.34±10.97	50.45±20.80	50.89±18.47	62.27±22.86	64.67±18.19	74.43±9.49	**75.23±17.63**
9	97.71±0.72	94.26±2.61	99.14±1.79	99.20±0.72	87.92±16.25	86.37±14.19	99.61±0.56	**99.68±0.36**
10	48.60±16.78	67.20±9.25	86.96±8.16	84.97±17.32	80.68±10.46	77.99±16.09	85.76±8.07	**91.01±5.75**
11	86.58±4.17	84.92±9.48	**98.34±1.74**	98.10±0.92	93.27±4.61	92.07±12.06	97.38±3.44	93.31±7.69
12	85.17±11.96	97.14±2.97	99.71±0.71	96.00±10.17	93.74±5.10	95.15±9.54	**99.80±0.24**	98.88±2.50
13	98.19±1.01	96.89±5.92	99.29±1.02	99.64±0.82	95.24±4.67	96.59±3.79	99.20±0.39	**99.69±0.62**
14	90.40±1.74	92.11±2.27	97.04±2.17	96.44±2.82	90.70±5.81	96.39±3.99	99.08±0.81	**99.19±0.71**
15	57.76±17.57	65.62±16.13	75.21±20.27	73.68±18.15	65.24±20.33	68.51±24.45	73.40±7.89	**84.88±8.32**
16	74.05±12.96	86.84±11.70	93.71±4.92	94.64±6.60	80.48±15.29	92.39±5.99	88.75±7.36	**97.84±1.90**
OA(%)	76.04±3.12	78.78±2.55	84.63±3.28	83.96±2.10	80.78±1.98	83.06±4.08	88.86±1.95	**90.94±3.53**
AA(%)	82.96±1.52	85.49±2.00	93.00±1.34	92.10±1.74	87.76±2.20	89.66±3.21	93.69±1.14	**94.75±1.41**
Kappa(×100)	73.49±3.36	76.50±2.78	83.00±3.56	82.25±2.29	78.71±2.16	81.22±4.55	87.63±2.15	**89.95±3.87**

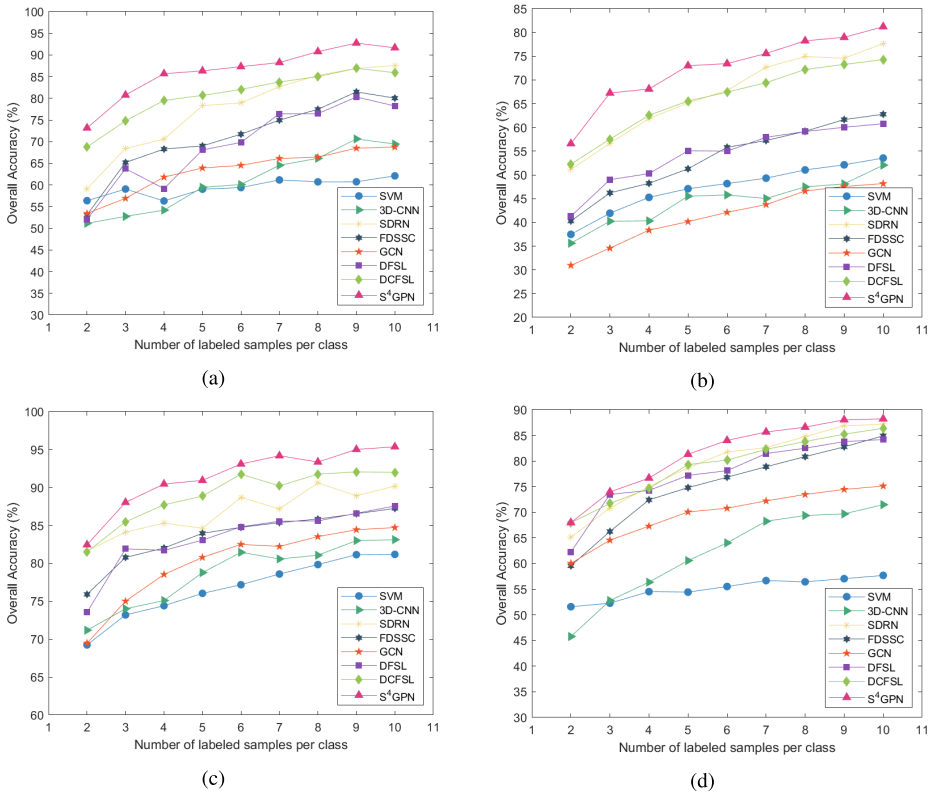


Fig. 11. Overall accuracies of various methods under different numbers of labeled samples per class. (a) PU dataset. (b) IP dataset. (c) Salinas dataset. (d) Houston dataset.

balanced manner. In addition, compared with the second best performing DCFSL method, our proposed S^{4}GPN improves 7.44% on OA, 7.14% on AA, and 0.0845 on Kappa, which indicates that the multistage feature extractor method achieves better classification results than the single-stage method. Fig. 8 indicates that the method combining GCN extracted spectral features can better deal with the boundary information than the DCFSL, DFSL, FDSSC, and SDRN methods.

3) Results on the Houston Dataset: In Table IX, the classification results for the Houston dataset are listed, and the best results are highlighted in bold. As shown in Table IX, deep learning methods (3D-CNN, SDRN, FDSSC) outperform traditional methods (SVM). In the DFSL and DCFSL methods, where FSL is introduced, higher classification performance is achieved with limited labeled samples. For example, the DCFSL method improves OA by 0.66% over the SDRN method and by 4.50% over the FDSSC method. Similarly, our proposed S^{4}GPN not only has the advantage of FSL but also obtains additional semantic information through the SSL processing of data augmentation on spectral and spatial data. As a result, compared with the other FSL methods, our method outperforms in terms of OA, AA, and Kappa. In addition, the

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 02:46:12 UTC from IEEE Xplore. Restrictions apply.

TABLE XI
OA(%) IN DIFFERENT COMBINATIONS

Data sets	PN	PN+SSL	PN+SSL+Fusion
PU	85.51±4.68	85.32±5.49	**86.34±3.38**
IP	69.43±3.29	69.65±2.75	**73.00±3.17**
Salinas	89.49±3.31	89.25±3.51	**90.94±3.53**
Houston	79.50±1.62	79.25±2.13	**81.32±1.51**

inclusion of GCN is beneficial in dealing with the classification boundary problem. As shown in Fig. 9, the S^{4}GPN method handles the boundary information better.

4) Results on the Salinas Dataset: Table X lists the results of our S^{4}GPN method and others for the Salinas dataset, where the best results are indicated in bold. As shown in Table X, our method improves the OA by 6.98% over the FDSSC method, by 10.16% over the GCN method, and by 2.08% over the DCFSL method, indicating that the spectral–spatial separation feature extractor combining GCN and Densenet can more fully exploit the feature information to further improve the classification accuracy compared with other spectral–spatial joint feature extractors. The visualization results are shown in Fig. 10, and it is observed that the GCN-based model has a much smoother visual effect.

To further verify that our method is effective, we analyze the classification performance of our proposed method and other compared methods under different labeled samples. We set the number of labeled samples per class from two to ten to conduct experiments on four datasets and measure their classification performance with OA. From Fig. 11, it can be observed that all the methods improve in classification performance as the number of training samples increases. Moreover, our proposed method has higher OA than other methods for all different numbers of labeled samples and remains relatively stable with the change in labeled samples on all four datasets.

E. Ablation Study

In this section, we explore the impact of all three stages of our S^{4}GPN. Since our proposed method consists of three stages, we set up three combinations to test the corresponding performances, i.e., PN, PN+SSL, and PN+SSL+Fusion. From Table XI, we find that the classification performance of PN+SSL degrades slightly compared with PN, which indicates that it is difficult for supervised learning to learn effective representations with limited samples and that SSL through data augmentation negatively affects the classification accuracy. However, the accuracy of PN+SSL+Fusion is significantly improved, which proves that the additional semantic information obtained in the SSL stage can help supervised learning to improve the feature representation and that effective feature fusion can further improve model performance.

F. Running Time Comparison

We record the time for training and testing of our S^{4}GPN and other compared methods. Notably, all the methods are implemented in the same environment and each method takes five labeled samples for the experiment. From Table XII, we can see that the time consumption of the FSL methods is more than others because FSL methods require more training epochs to yield higher accuracy. Among the compared methods, our method takes a shorter time than DFSL and DCFSL because our method uses a more simplified feature extractor and does not require transfer learning, but takes a longer time than other methods since our method has three stages.

TABLE XII
TRAINING AND TESTING TIME ON FOUR DATASETS

		PU	IP	Salinas	Houston
3D-CNN	Training time(s)	53.23	68.74	69.35	62.46
	Testing time(s)	4.46	1.69	4.46	3.69
SDRN	Training time(s)	69.33	157.20	161.83	84.67
	Testing time(s)	3.07	4.90	16.74	5.27
FDSSC	Training time(s)	168.70	312.24	313.64	286.54
	Testing time(s)	9.29	3.94	21.33	4.30
GCN	Training time(s)	28.74	18.74	75.57	39.61
	Testing time(s)	3.21	2.05	7.83	3.65
DFSL	Training time(s)	435.78	711.48	965.90	616.30
	Testing time(s)	5.16	1.30	6.09	3.07
DCFSL	Training time(s)	1216.32	1263.95	1326.49	920.98
	Testing time(s)	6.78	1.61	7.63	11.81
S^{4}GPN	Training time(s)	424.18	528.16	513.58	497.47
	Testing time(s)	9.34	3.95	6.83	9.96

V. CONCLUSION

In this article, we have introduced a novel self-supervised FSL method for HIC. Different from prior works that only use FSL to solve small sample classification problems, our method uses SSL and uses the supervised information obtained from pretraining learning to guide SSL. We further integrate the information obtained from pre-training and SSL to broaden and enrich the original representation. Moreover, in the SSL stage, we propose a novel approach to data augmentation that uses different properties of spectral and spatial information of HSIs to enhance the model generalization capability. In addition, all three stages of our method use the same spectral–spatial separation feature extraction structure, and each training stage is conducive to model performance improvement. By conducting experiments on four datasets, the results show that our method leads to a significant performance improvement. In future works, we will further explore SSL methods to help address the problem of limited labeled samples.

REFERENCES

[1] P. Ghamisi et al., "Advances in hyperspectral image and signal processing: A comprehensive overview of the state of the art," *IEEE Geosci. Remote Sens. Mag.*, vol. 5, no. 4, pp. 37–78, Dec. 2017.

[2] I. B. Strachan, E. Pattey, and J. B. Boisvert, "Impact of nitrogen and environmental conditions on corn as detected by hyperspectral reflectance," *Remote Sens. Environ.*, vol. 80, no. 2, pp. 213–224, May 2002.

[3] P. Kuflik and S. R. Rotman, "Band selection for gas detection in hyperspectral images," in *Proc. IEEE 27th Conv. Electr. Electron. Eng. Isr.*, Nov. 2012, pp. 1–4.

[4] M. Awad, "Sea water chlorophyll—A estimation using hyperspectral images and supervised artificial neural network," *Ecol. Informat.*, vol. 24, pp. 60–68, Nov. 2014.

[5] S. Jay and M. Guillaume, "A novel maximum likelihood based method for mapping depth and water quality from hyperspectral remote-sensing data," *Remote Sens. Environ.*, vol. 147, pp. 121–132, May 2014.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 02:46:12 UTC from IEEE Xplore. Restrictions apply.

[6] X. Yang and Y. Yu, "Estimating soil salinity under various moisture conditions: An experimental study," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 5, pp. 2525–2533, May 2017.

[7] S. A. E. Rahman and A. H. S. Zolait, "Hyperspectral image analysis for oil spill detection: A comparative study," *Int. J. Comput. Sci. Math.*, vol. 9, no. 2, p. 103, 2018.

[8] P. C. Pandey, P. K. Srivastava, H. Balzter, B. Bhattacharya, and G. P. Petropoulos, "Hyperspectral remote sensing in precision agriculture: Present status, challenges, and future trends," *Hyperspectral Remote Sens.*, vol. 2020, pp. 121–146, Jan. 2020.

[9] J.-H. Yang, L.-G. Wang, and J.-X. Qian, "Hyperspectral image classification based on spatial and spectral features and sparse representation," *Appl. Geophys.*, vol. 11, no. 4, pp. 489–499, Dec. 2014.

[10] M. Fauvel, Y. Tarabalka, J. A. Benediktsson, J. Chanussot, and J. C. Tilton, "Advances in spectral–spatial classification of hyperspectral images," *Proc. IEEE*, vol. 101, no. 3, pp. 652–675, Mar. 2013.

[11] N. Audebert, B. Le Saux, and S. Lefevre, "Deep learning for classification of hyperspectral data: A comparative review," *IEEE Geosci. Remote Sens. Mag.*, vol. 7, no. 2, pp. 159–173, Jun. 2019.

[12] F. Melgani and L. Bruzzone, "Classification of hyperspectral remote sensing images with support vector machines," *IEEE Trans. Geosci. Remote Sens.*, vol. 42, no. 8, pp. 1778–1790, Aug. 2004.

[13] B.-C. Kuo, H.-H. Ho, C.-H. Li, C.-C. Hung, and J.-S. Taur, "A kernel-based feature selection method for SVM with RBF kernel for hyperspectral image classification," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 7, no. 1, pp. 317–326, Jan. 2014.

[14] P. K. Murphy and M. A. Kolodner, "Implementation of a multiscale Bayesian classification approach for hyperspectral terrain categorization," *Proc. SPIE*, vol. 4816, pp. 278–287, Nov. 2002.

[15] J. Ham, Y. Chen, M. M. Crawford, and J. Ghosh, "Investigation of the random forest framework for classification of hyperspectral data," *IEEE Trans. Geosci. Remote Sens.*, vol. 43, no. 3, pp. 492–501, Mar. 2005.

[16] W. Li, C. Chen, H. Su, and Q. Du, "Local binary patterns and extreme learning machine for hyperspectral imagery classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 53, no. 7, pp. 3681–3693, Jul. 2015.

[17] B. Cui, J. Cui, Y. Lu, N. Guo, and M. Gong, "A sparse representation-based sample pseudo-labeling method for hyperspectral image classification," *Remote Sens.*, vol. 12, no. 4, p. 664, Feb. 2020.

[18] J. Chen and R. S. Wang, "Applied research of Gaussian maximum likelihood classification in hyperspectral classification," *J. Comput. Appl.*, vol. 26, no. 8, pp. 1876–1878, 2006.

[19] M. Khodadadzadeh, R. Rajabi, and H. Ghassemian, "A novel approach for spectral–spatial classification of hyperspectral data based on SVM-MRF method," in *Proc. IEEE Int. Geosci. Remote Sens. Symp.*, Jul. 2011, pp. 1890–1893.

[20] L. Sun et al., "Low rank component induced spatial–spectral kernel method for hyperspectral image classification," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 30, no. 10, pp. 3829–3842, Oct. 2020.

[21] L. Pan, C. He, Y. Xiang, and L. Sun, "Multiscale adjacent superpixel-based extended multi-attribute profiles embedded multiple kernel learning method for hyperspectral classification," *Remote Sens.*, vol. 13, no. 1, p. 50, Dec. 2020.

[22] C. Zhao, X. Wan, G. Zhao, and Y. Yan, "Spectral–spatial classification of hyperspectral images using trilateral filter and stacked sparse autoencoder," *J. Appl. Remote Sens.*, vol. 11, no. 1, pp. 16–33, 2017.

[23] Y. Chen, X. Zhao, and X. Jia, "Spectral–spatial classification of hyperspectral data based on deep belief network," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 8, no. 6, pp. 2381–2392, Jun. 2015.

[24] Y. Chen, L. Zhu, P. Ghamisi, X. Jia, G. Li, and L. Tang, "Hyperspectral images classification with Gabor filtering and convolutional neural network," *IEEE Geosci. Remote Sens. Lett.*, vol. 14, no. 12, pp. 2355–2359, Dec. 2017.

[25] Y. Li, H. Zhang, and Q. Shen, "Spectral–spatial classification of hyperspectral imagery with 3D convolutional neural network," *Remote Sens.*, vol. 9, no. 1, p. 67, Jan. 2017.

[26] S. Li, X. Zhu, Y. Liu, and J. Bao, "Adaptive spatial–spectral feature learning for hyperspectral image classification," *IEEE Access*, vol. 7, pp. 61534–61547, 2019.

[27] Y. Ma, Z. Liu, and C. L. P. Chen Chen, "Hybrid spatial–spectral feature in broad learning system for hyperspectral image classification," *Int. J. Speech Technol.*, vol. 52, no. 3, pp. 2801–2812, Feb. 2022.

[28] Z. Gao, L. Tong, J. Zhou, B. Qian, J. Yu, and C. Xiao, "Stochastic depth residual network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5506113.

[29] W. Cai et al., "A novel hyperspectral image classification model using Bole convolution with three-direction attention mechanism: Small sample and unbalanced learning," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 5500917.

[30] W. Zhao and S. Du, "Spectral–spatial feature extraction for hyperspectral image classification: A dimension reduction and deep learning approach," *IEEE Trans. Geosci. Remote Sens.*, vol. 54, no. 8, pp. 4544–4554, Aug. 2016.

[31] W. Wang, S. Dou, Z. Jiang, and L. Sun, "A fast dense spectral–spatial convolution network framework for hyperspectral images classification," *Remote Sens.*, vol. 10, no. 7, p. 1068, Jul. 2018.

[32] L. Chen, Z. Wei, and Y. Xu, "A lightweight spectral–spatial feature extraction and fusion network for hyperspectral image classification," *Remote Sens.*, vol. 12, no. 9, p. 1395, Apr. 2020.

[33] A. Qin, Z. Shang, J. Tian, Y. Wang, T. Zhang, and Y. Y. Tang, "Spectral–spatial graph convolutional networks for semisupervised hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 16, no. 2, pp. 241–245, Feb. 2019.

[34] L. Mou, X. Lu, X. Li, and X. X. Zhu, "Nonlocal graph convolutional networks for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 12, pp. 8246–8257, Dec. 2020.

[35] B. Liu, K. Gao, A. Yu, W. Guo, R. Wang, and X. Zuo, "Semisupervised graph convolutional network for hyperspectral image classification," *J. Appl. Remote Sens.*, vol. 14, no. 2, p. 1, Jun. 2020.

[36] D. Hong, L. Gao, J. Yao, B. Zhang, A. Plaza, and J. Chanussot, "Graph convolutional networks for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 7, pp. 5966–5978, Jul. 2021.

[37] B. Xi et al., "Semisupervised cross-scale graph prototypical network for hyperspectral image classification," *IEEE Trans. Neural Netw. Learn. Syst.*, early access, Mar. 23, 2022, doi: 10.1109/TNNLS.2022.3158280.

[38] O. Vinyals, C. Blundell, T. Lillicrap, K. Kavukcuoglu, and D. Wierstra, "Matching networks for one shot learning," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 36, 2016, pp. 37–45.

[39] B. Liu, X. Yu, A. Yu, P. Zhang, G. Wan, and R. Wang, "Deep few-shot learning for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 4, pp. 2290–2304, Apr. 2019.

[40] H. Tang, Y. Li, X. Han, Q. Huang, and W. Xie, "A spatial–spectral prototypical network for hyperspectral remote sensing image," *IEEE Geosci. Remote Sens. Lett.*, vol. 17, no. 1, pp. 167–171, Jan. 2020.

[41] Y. Wang, Q. Yao, J. T. Kwok, and L. M. Ni, "Generalizing from a few examples: A survey on few-shot learning," *ACM Comput. Surv.*, vol. 53, no. 3, pp. 1–34, May 2021.

[42] Z. Li, M. Liu, Y. Chen, Y. Xu, W. Li, and Q. Du, "Deep cross-domain few-shot learning for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5501618.

[43] L. Jing and Y. Tian, "Self-supervised visual feature learning with deep neural networks: A survey," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 43, no. 11, pp. 4037–4058, Nov. 2021.

[44] X. Liu et al., "Self-supervised learning: Generative or contrastive," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 1, pp. 857–876, Jan. 2023.

[45] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, "A simple framework for contrastive learning of visual representations," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 1597–1607.

[46] J. B. Grill et al., "Bootstrap your own latent: A new approach to self-supervised learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 1–12.

[47] X. Hu, T. Li, T. Zhou, and Y. Peng, "Deep spatial–spectral subspace clustering for hyperspectral images based on contrastive learning," *Remote Sens.*, vol. 13, no. 21, p. 4418, Nov. 2021.

[48] X. Chen and K. He, "Exploring simple Siamese representation learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2021, pp. 15745–15753.

[49] J. Yue, L. Fang, H. Rahmani, and P. Ghamisi, "Self-supervised learning with adaptive distillation for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5501813.

[50] J. Snell, K. Swersky, and R. S. Zemel, "Prototypical networks for few-shot learning," in *Proc. 31st Int. Conf. Neural Inf. Process. Syst.*, 2017, pp. 4080–4090.

[51] A. Kolesnikov, X. Zhai, and L. Beyer, "Revisiting self-supervised visual representation learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 1920–1929.

[52] T. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. Int. Conf. Learn. Represent.*, 2016, pp. 1–6.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 02:46:12 UTC from IEEE Xplore. Restrictions apply.



Shan Ma received the B.S. degree from Shandong Normal University, Shandong, China, in 2017. She is currently pursuing the M.S. degree with the Faculty of Information Technology, Beijing University of Technology, Beijing, China.

Her research interests include pattern recognition and computer vision, and image processing.



Lei Tong (Member, IEEE) received the B.E. degree in measurement and control technology and instrumentation, the M.E. degree in measurement technology and automation devices from Beijing Jiaotong University, Beijing, China, in 2010 and 2012, respectively, and the Ph.D. degree in engineering from Griffith University, Brisbane, Australia, in 2016.

Currently, he is an Associate Professor with the Faculty of Information Technology, Beijing University of Technology, Beijing. His research interests include signal and image processing, pattern recognition, and remote sensing.



Jun Zhou (Senior Member, IEEE) received the B.S. degree in computer science and the B.E. degree in international business from the Nanjing University of Science and Technology, Nanjing, China, in 1996 and 1998, respectively, the M.S. degree in computer science from Concordia University, Montreal, QC, Canada, in 2002, and the Ph.D. degree in computing science from the University of Alberta, Edmonton, AB, Canada, in 2006.

He was a Research Fellow with the Research School of Computer Science, Australian National University, Canberra, ACT, Australia, and a Researcher with the Canberra Research Laboratory, NICTA, Canberra, ACT, Australia. In June 2012, he joined the School of Information and Communication Technology, Griffith University, Nathan, QLD, Australia, where he is currently a Professor. His research interests include pattern recognition, computer vision, and spectral imaging with their applications in remote sensing and environmental informatics.



Jing Yu received the Ph.D. degree in electronic engineering from Tsinghua University, Beijing, China, in 2011.

She is currently an Associate Professor with the Faculty of Information Technology, Beijing University of Technology, Beijing. Her research interests include image processing and pattern recognition.



Chuangbai Xiao received the Ph.D. degree in Department of Automation, Tsinghua University, Beijing, China, in 1995.

Currently, he is a Professor with the Faculty of Information Technology, Beijing University of Technology, Beijing. His research interests include pattern recognition, digital image processing, digital signal processing, computer network security, and cloud computing.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 02:46:12 UTC from IEEE Xplore. Restrictions apply.